

ECOTRACK

CAMPUS SUSTAINABILITY ISSUE MANAGEMENT SYSTEM

PROJECT REPORT

Submitted by

AKSHITHA S S

B.Tech – Information Technology

Technologies Used

HTML | CSS | JavaScript | Java | Spring Boot | MySQL

2026

1. PROJECT OVERVIEW

1. PROJECT OVERVIEW

EcoTrack is a Campus Sustainability Issue Management System developed to help students and campus staff report and manage sustainability-related issues efficiently. The system allows users to submit issues related to areas such as electricity, water, waste management, cleanliness, and other campus facilities.

The application provides a simple web interface for creating, viewing, updating, deleting, searching, and filtering reports. The backend is developed using Java and Spring Boot, with REST APIs used for communication between the frontend and backend. MySQL is used to store the reported issues, while Spring Data JPA and Hibernate handle database operations.

EcoTrack demonstrates a complete CRUD-based web application with frontend integration, database connectivity, server-side validation, exception handling, and API testing.

2. PROBLEM STATEMENT

In a campus environment, sustainability-related problems such as water leakage, electricity wastage, improper waste disposal, and cleanliness issues may not be reported or tracked systematically. Manual reporting methods can make it difficult to monitor issues, update their status, and maintain records.

EcoTrack addresses this problem by providing a centralized web-based system where campus sustainability issues can be reported, stored, tracked, updated, and resolved efficiently.

3. OBJECTIVES

- To develop a web-based system for reporting campus sustainability issues.
- To implement complete CRUD operations for issue management.
- To provide REST APIs for communication between frontend and backend.
- To store and manage issue data using MySQL.
- To implement server-side validation and exception handling.
- To provide search and filtering functionality for easier issue tracking.
- To test the application using Postman and database verification.

4. TECHNOLOGIES USED

Technology	Purpose
HTML	Provides the structure of the web interface
CSS	Provides styling and responsive design
JavaScript	Handles frontend interactions and API communication
Java	Programming language used for backend development
Spring Boot	Develops the backend and REST APIs
Spring Data JPA	Simplifies database operations using ORM
Hibernate	Handles object-relational mapping
MySQL	Stores and manages application data
Postman	Used for testing REST APIs
Git & GitHub	Used for version control and project management
IntelliJ IDEA	Used for Spring Boot development
VS Code	Used for frontend development
MySQL Workbench	Used for database management

5. SYSTEM ARCHITECTURE

5. SYSTEM ARCHITECTURE

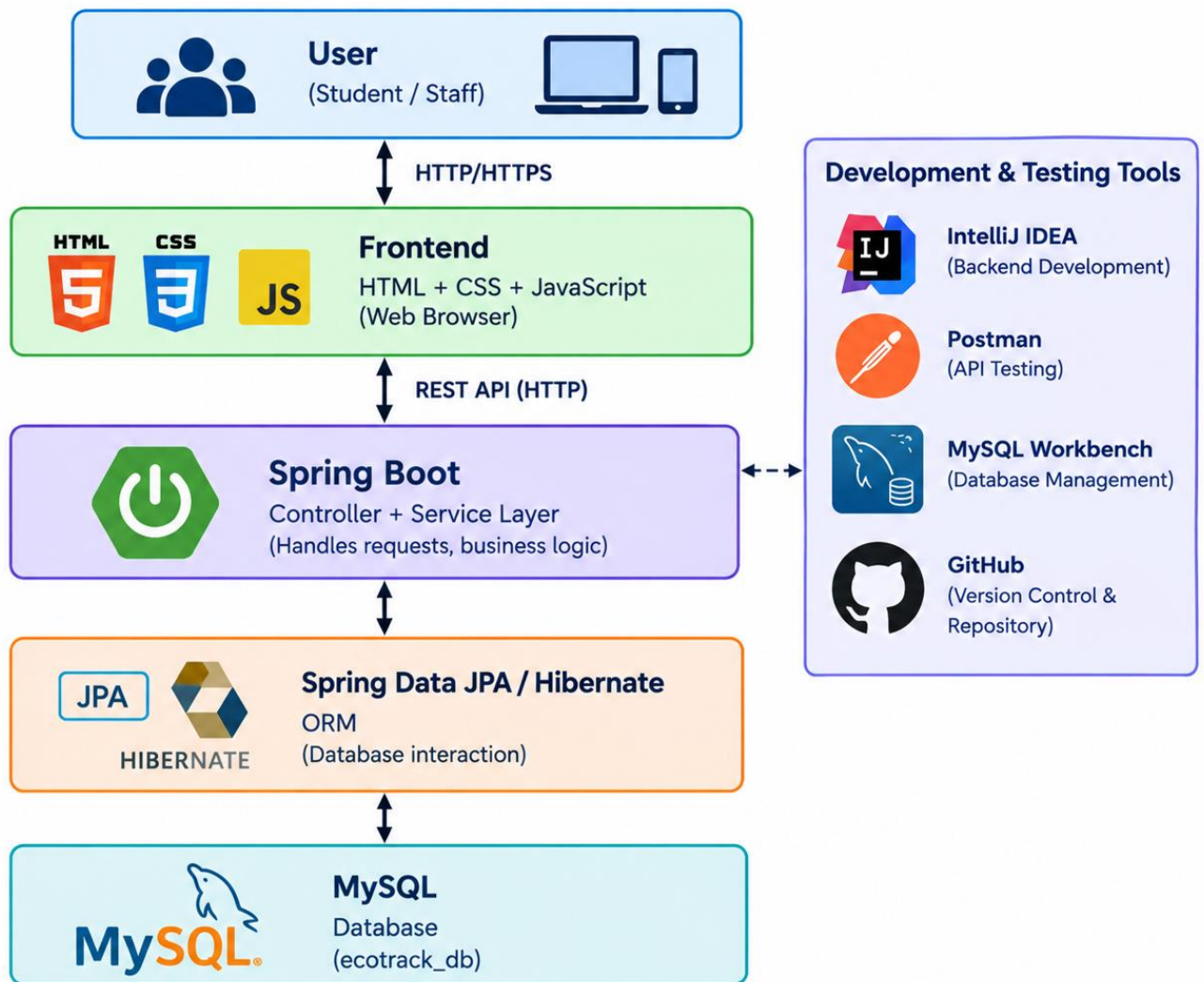


Figure 1: System Architecture of EcoTrack

6. DATABASE DESIGN – ER DIAGRAM

6.1 Database Overview

EcoTrack uses **MySQL** as its relational database management system. The database is named `ecotrack_db` and stores information about campus sustainability issues reported by users.

The main table used by the application is **eco_reports**. Each report is uniquely identified by an automatically generated ID. The table stores the reporter's details, issue location, category, description, priority, status, and the date and time at which the report was created.

6.2 Entity: Eco Report

The `eco_reports` entity represents a sustainability issue reported on the campus.

Attribute	Description
id	Primary key that uniquely identifies each report
reporter_name	Name of the person reporting the issue
location	Campus location where the issue occurred
category	Type of sustainability issue
description	Detailed description of the reported issue
priority	Priority level of the issue
status	Current status of the report
created_at	Date and time when the report was created

6.3 ER Diagram

6.3 ER Diagram

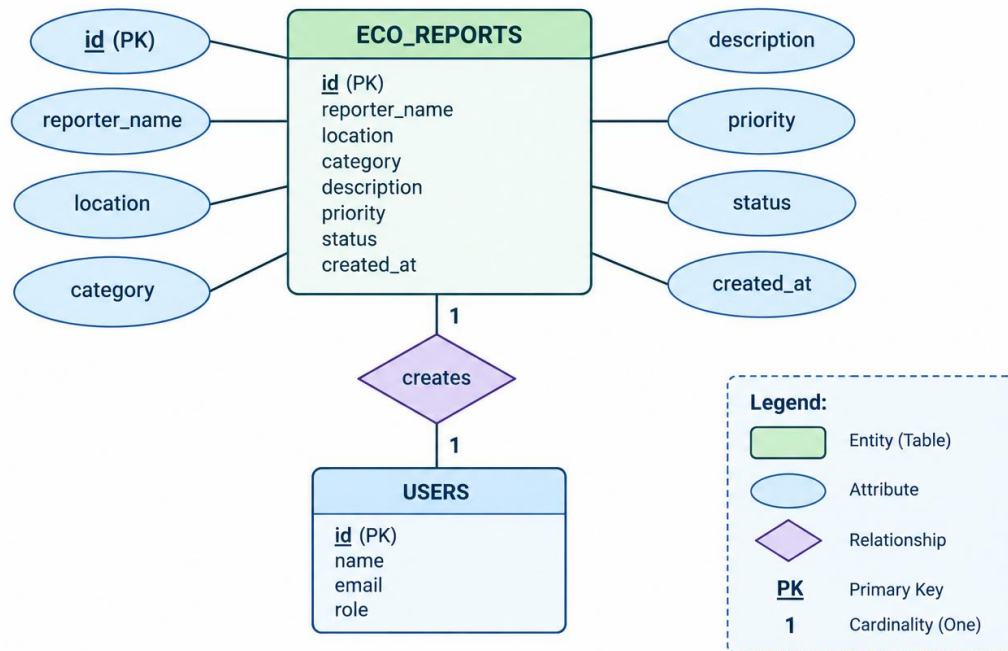


Figure 2: Entity Relationship Diagram of EcoTrack

6.4 Database Design Explanation

The `eco_reports` table is designed to maintain all sustainability issue records in a structured format. The `id` field acts as the **Primary Key**, ensuring that every report has a unique identifier.

The application uses **Spring Data JPA and Hibernate** to map the Java `EcoReport` entity to the MySQL `eco_reports` table. This allows the application to perform database operations such as creating, retrieving, updating, and deleting reports without writing SQL queries for every operation.

The database design supports the core CRUD functionality of the EcoTrack application and provides persistent storage for all reported sustainability issues.

7. CRUD OPERATIONS

7. CRUD OPERATIONS

EcoTrack implements complete **CRUD (Create, Read, Update, Delete)** functionality for managing campus sustainability issue reports. These operations are provided through REST APIs developed using Spring Boot.

7.1 Create

The Create operation allows users to submit a new sustainability issue by entering the reporter name, location, category, description, priority, and status.

API: POST /api/reports

A new report is stored in the MySQL database with a unique ID and creation timestamp.

7.2 Read

The Read operation allows users to retrieve existing sustainability reports from the database.

API: GET /api/reports

The system also supports retrieving an individual report using its ID.

API: GET /api/reports/{id}

7.3 Update

The Update operation allows an existing report to be modified. Users can update details such as location, category, description, priority, and status.

API: PUT /api/reports/{id}

The updated information is saved back to the MySQL database.

7.4 Delete

The Delete operation allows an existing sustainability report to be removed from the system.

API: DELETE /api/reports/{id}

After successful deletion, the system returns a confirmation message.

7.5 Search and Filtering

EcoTrack also provides additional functionality for easier issue management:

- Search reports by location.
- Filter reports by category.
- Filter reports by status.
- Filter reports by priority.

These features help users quickly locate and manage specific sustainability issues.

8. API DOCUMENTATION

EcoTrack provides RESTful APIs using Spring Boot to perform CRUD operations and additional search and filtering operations. The APIs communicate with the frontend and interact with the MySQL database through Spring Data JPA and Hibernate.

8.1 API Endpoints

HTTP Method	Endpoint	Description
POST	/api/reports	Creates a new sustainability report
GET	/api/reports	Retrieves all sustainability reports
GET	/api/reports/{id}	Retrieves a specific report by ID
PUT	/api/reports/{id}	Updates an existing report
DELETE	/api/reports/{id}	Deletes a report by ID
GET	/api/reports/search?location={location}	Searches reports by location
GET	/api/reports/category/{category}	Filters reports by category
GET	/api/reports/status/{status}	Filters reports by status
GET	/api/reports/priority/{priority}	Filters reports by priority

8.2 Request and Response

The API uses **JSON (JavaScript Object Notation)** for sending request data and receiving responses.

Sample POST Request:

```
{
  "reporterName": "Priya",
  "location": "Block B",
  "category": "Water",
  "description": "Water leakage near laboratory",
}
```

```
"priority": "Medium",  
"status": "Pending"}
```

Sample Successful Response:

```
{  
  "id": 2,  
  "reporterName": "Priya",  
  "location": "Block B",  
  "category": "Water",  
  "description": "Water leakage near laboratory",  
  "priority": "Medium",  
  "status": "Pending",  
  "createdAt": "2026-09-15T09:00:00"  
}
```

8.3 HTTP Status Codes

Status Code	Meaning	Usage in EcoTrack
200 OK	Request successful	GET, PUT and DELETE operations
201 Created	Resource successfully created	POST operation
400 Bad Request	Invalid request data	Validation failure
404 Not Found	Resource does not exist	Invalid report ID
500 Internal Server Error	Unexpected server error	Server-side failure

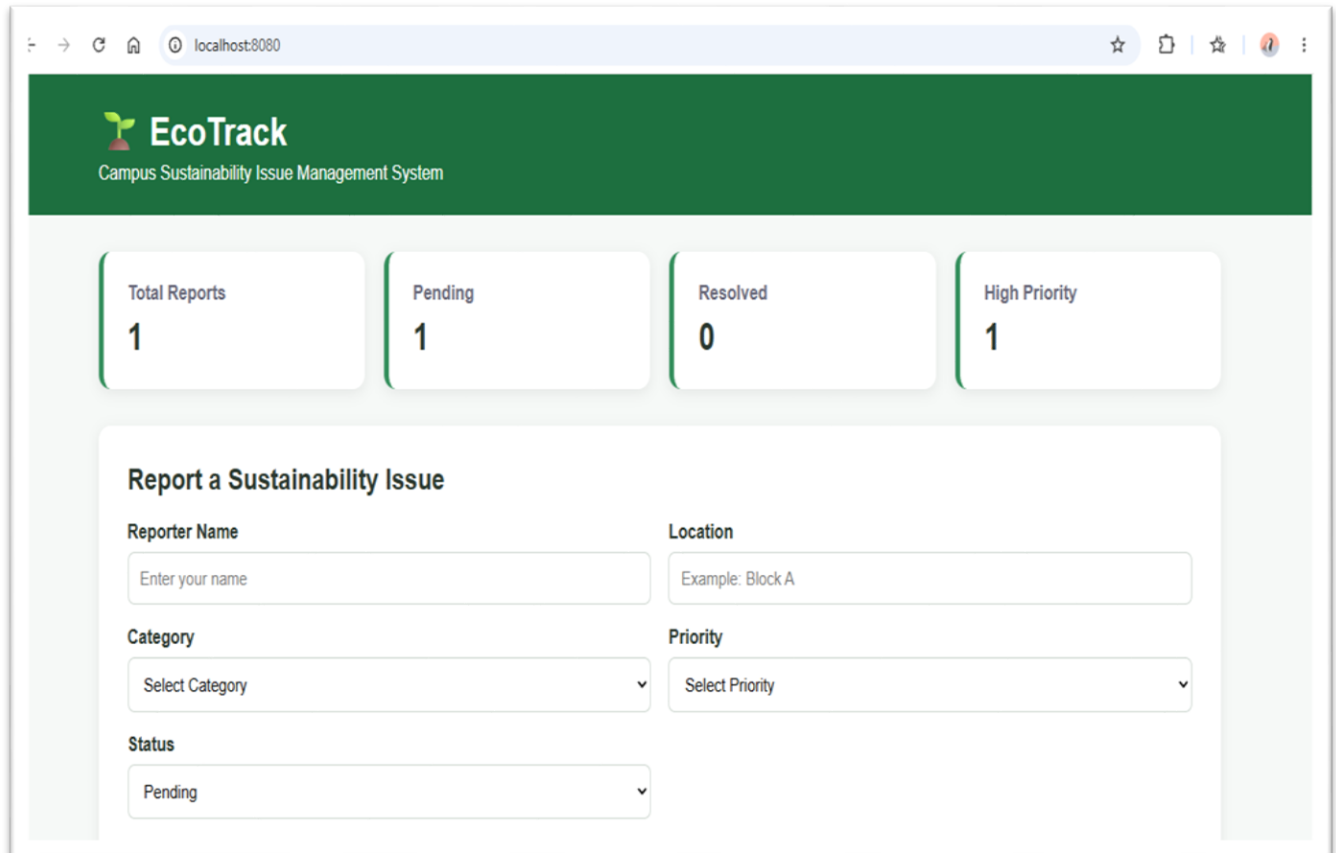
8.4 API Testing Tool

The EcoTrack REST APIs were tested using **Postman**. Valid requests, invalid input validation, CRUD operations, and non-existing resource handling were tested to verify that the backend behaves correctly.

9. APPLICATION SCREENSHOTS

The following screenshots demonstrate the working EcoTrack application, including the user interface, report creation, report management, and database verification.

9.1 EcoTrack Dashboard



The screenshot displays the EcoTrack application interface in a web browser. The browser's address bar shows 'localhost:8080'. The application header is a dark green bar with the 'EcoTrack' logo (a small green plant icon) and the text 'Campus Sustainability Issue Management System'.

Below the header, there are four white cards with green borders, each displaying a statistic:

- Total Reports:** 1
- Pending:** 1
- Resolved:** 0
- High Priority:** 1

Below these cards is a section titled 'Report a Sustainability Issue' containing a form with the following fields:

- Reporter Name:** A text input field with the placeholder 'Enter your name'.
- Location:** A text input field with the placeholder 'Example: Block A'.
- Category:** A dropdown menu with 'Select Category' as the selected option.
- Priority:** A dropdown menu with 'Select Priority' as the selected option.
- Status:** A dropdown menu with 'Pending' as the selected option.

Figure 3: EcoTrack Dashboard

9.2 Creating a Sustainability Report

Report a Sustainability Issue

Reporter Name:

Location:

Category:

Priority:

Status:

Description:

Figure 4: Creating a Sustainability Report

9.3 Report Display and Management

Campus Reports

ID	Reporter	Location	Category	Description	Priority	Status	Actions
1	Arun	Block A	Electricity	LIGHTS ARE NOT WORKING PROPERLY	High	Pending	Edit Delete

Figure 5: Report Display and Management

9.4 Search and Filtering

Update Sustainability Report

Reporter Name
Arun

Location
Block A

Category
Electricity

Priority
High

Status
In Progress

Description
LIGHTS ARE NOT WORKING PROPERLY

Update Report **Cancel**

Search by location... All Categories All Status **Reset**

Figure 6: Search and Filtering of Reports

9.5 Database Verification

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: Server Status, Client Connections, Users and Privileges, Status and System Variables, Data Export, Data Import/Restore

INSTANCE: Startup / Shutdown, Server Logs, Options File

PERFORMANCE: Dashboard, Performance Reports, Performance Schema Setup

Administration Schemas

Information: No object selected

SQL File 18*:
1. CREATE DATABASE ecotrack_db;
2. USE ecotrack_db;
3. USE ecotrack_db;
4. SELECT * FROM eco_reports;

Result Grid:

#	id	category	created_at	description	location	priority	reporter_name	status
1	1	Electricity	2026-09-15 03:21:20.185667	LIGHTS ARE NOT WORKING P...	Block A	High	Arun	Pending

Output:

#	Time	Action	Message	Duration / Fetch
1	16:59:05	CREATE DATABASE ecotrack_db	1 row(s) affected	0.047 sec
2	16:59:08	USE ecotrack_db	0 row(s) affected	0.000 sec
3	08:54:06	USE ecotrack_db	0 row(s) affected	0.000 sec
4	08:54:11	SELECT * FROM eco_reports LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Figure 7: EcoTrack Database Verification

10. API TESTING

The EcoTrack REST APIs were tested using Postman to verify the correctness of CRUD operations, input validation, and exception handling. Both valid and invalid requests were tested.

10.1 Successful API Request – 200 OK

A valid API request was sent to the EcoTrack backend and the server successfully processed the request.

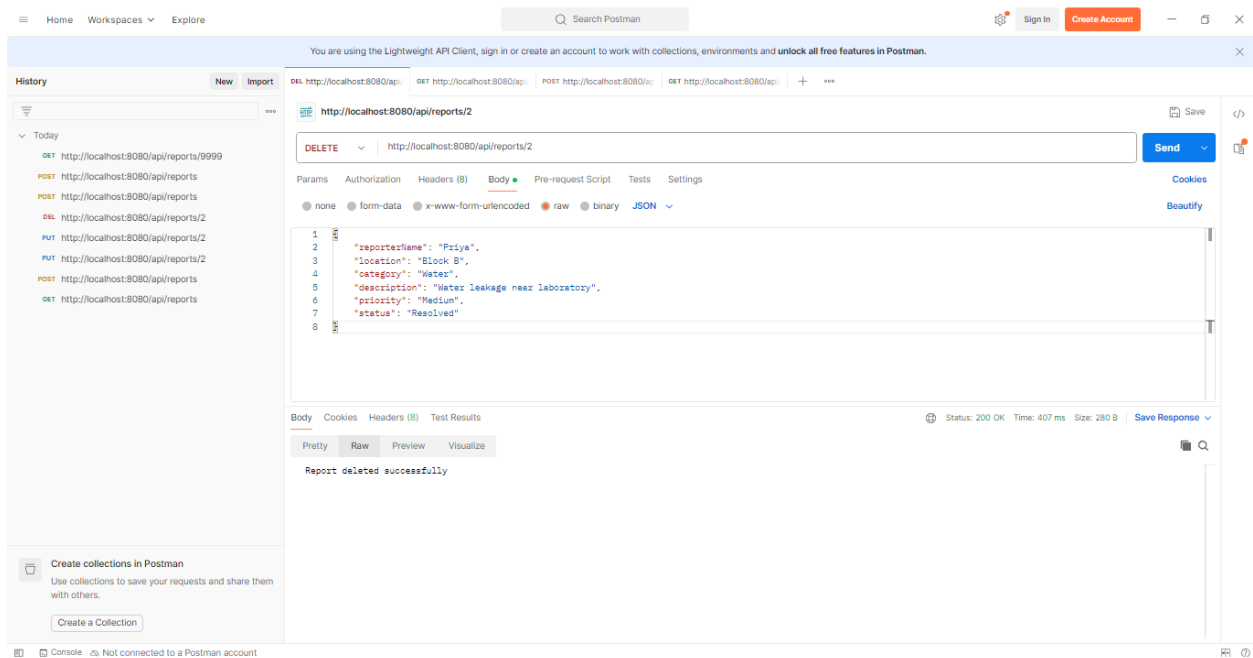


Figure 8: Successful API Request – 200 OK

10.2 Validation Testing – 400 Bad Request

Invalid input data was submitted with required fields left empty. The server-side validation correctly rejected the request and returned a **400 Bad Request** response with validation error messages.

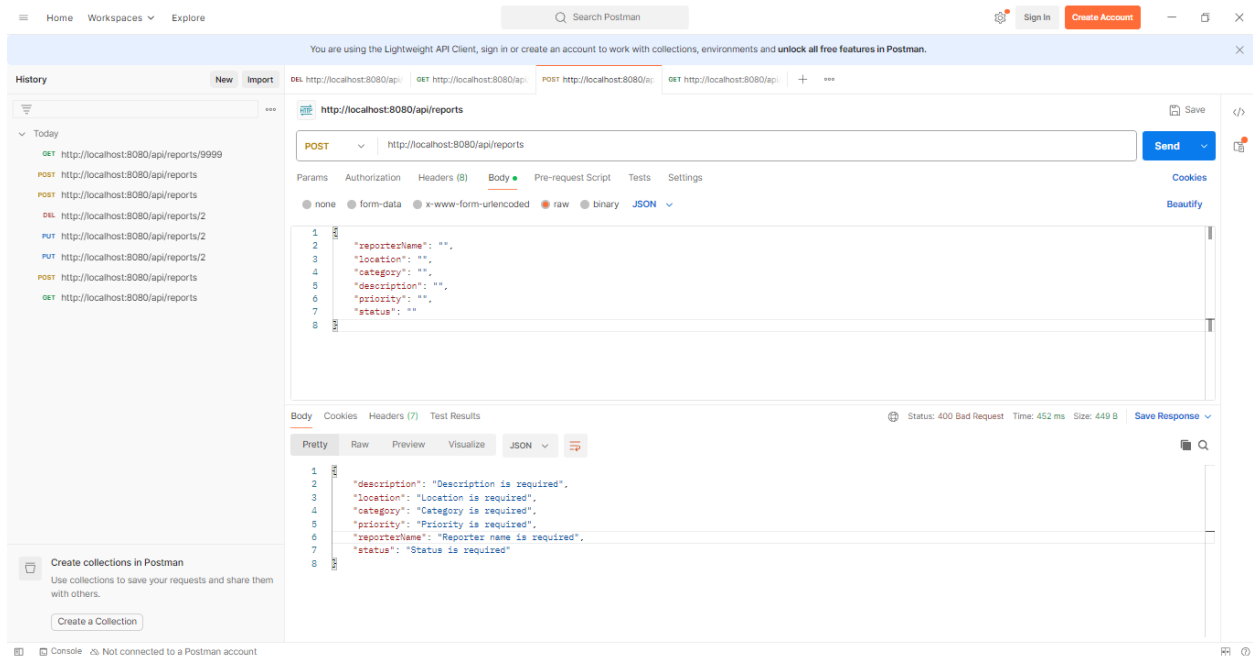


Figure 9: Validation Error – 400 Bad Request

10.3 Exception Testing – 404 Not Found

A request was sent for a report ID that does not exist in the database. The application correctly handled the exception and returned a **404 Not Found** response.

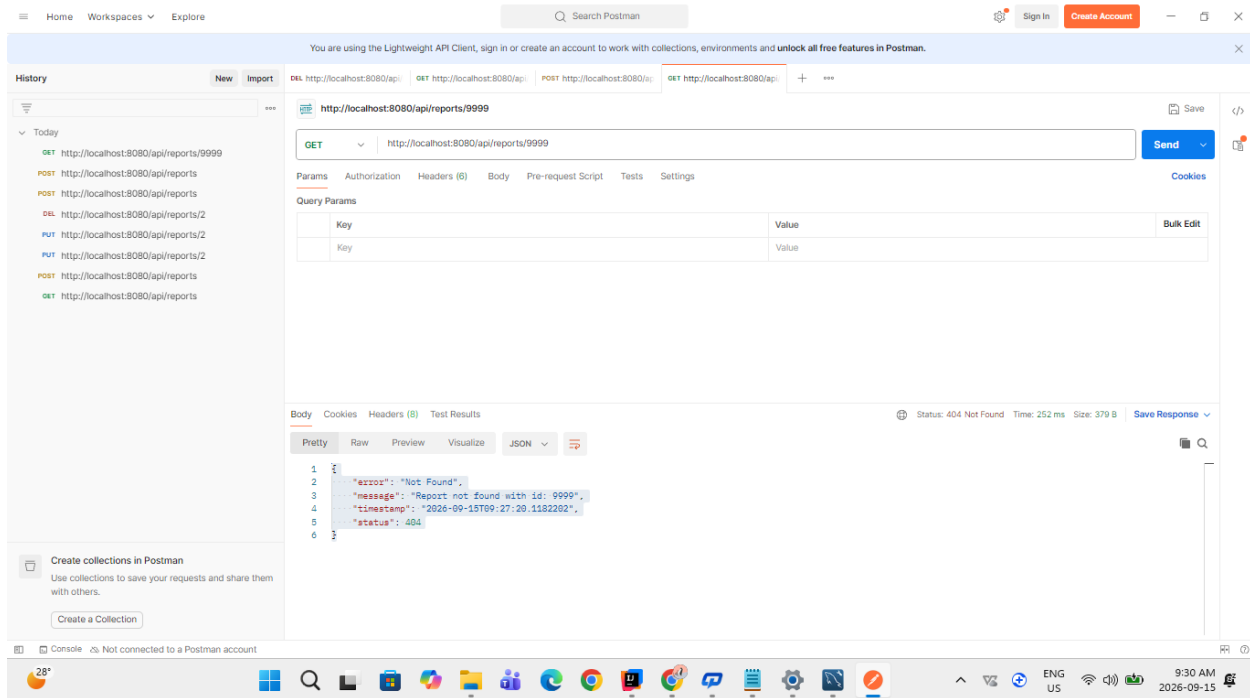


Figure 10: Resource Not Found – 404 Exception

10.4 Testing Summary

Test Case	Expected Result	Actual Result	Status
Create valid report	Report created successfully	Report created	PASS
Retrieve reports	Reports returned	Reports returned	PASS
Update report	Report updated	Report updated	PASS
Delete report	Report deleted	Report deleted	PASS
Invalid/empty input	400 Bad Request	400 Bad Request	PASS
Non-existing report ID	404 Not Found	404 Not Found	PASS

11. VALIDATION AND EXCEPTION HANDLING

EcoTrack implements server-side validation and exception handling to improve the reliability and correctness of the application.

11.1 Input Validation

The application validates important report fields before storing data in the database. Required fields are checked using validation annotations provided by Jakarta Bean Validation.

The following fields are required:

- Reporter Name
- Location
- Category
- Description
- Priority
- Status

If any required field is empty, the server rejects the request and returns a **400 Bad Request** response with an appropriate validation message.

11.2 Exception Handling

EcoTrack uses a global exception handling mechanism to provide meaningful responses when errors occur.

If a user requests a report ID that does not exist, the application throws a `ResourceNotFoundException` and returns a **404 Not Found** response.

The application also handles unexpected server-side errors and returns an appropriate **500 Internal Server Error** response.

11.3 Benefits

- Prevents incomplete data from being stored.
- Provides clear error messages to users.
- Handles invalid resource requests properly.

12. TOOLS AND DEVELOPMENT ENVIRONMENT

EcoTrack was developed using a combination of development, database, and API testing tools.

12.1 Development Tools

IntelliJ IDEA was used for developing the Java Spring Boot backend and managing the Maven project.

MySQL Workbench was used to create and manage the EcoTrack database and to verify stored report records.

Postman was used to test the REST API endpoints, including Create, Read, Update, Delete, validation, and exception handling operations.

Git and GitHub are used for version control and maintaining the project source code.

12.2 Technologies Used

- ☐ HTML for webpage structure
- ☐ CSS for styling and responsive design
- ☐ JavaScript for frontend interaction and API communication
- ☐ Java for backend development
- ☐ Spring Boot for REST API development
- ☐ Spring Data JPA and Hibernate for ORM and database operations
- ☐ MySQL for relational data storage
- ☐ Postman for API testing
- ☐ Git and GitHub for version control

12.3 Development Environment

The application was developed and tested in a local development environment.

Backend: Java with Spring Boot

Database: MySQL 8.0

Frontend: HTML, CSS and JavaScript

API Testing: Postman

Database Management: MySQL Workbench

IDE: IntelliJ IDEA

13. CHALLENGES FACED

During the development of EcoTrack, several technical and implementation challenges were encountered.

13.1 Backend and Database Integration

Connecting the Spring Boot application with MySQL and configuring Spring Data JPA required proper database and application configuration.

13.2 REST API Development

Creating and testing REST endpoints for different CRUD operations required proper HTTP methods, request formats, response handling, and endpoint mapping.

13.3 Input Validation

Handling incomplete or invalid report data was important to maintain data quality. Server-side validation was implemented to prevent empty required fields.

13.4 Exception Handling

Handling requests for non-existing report IDs required custom exception handling so that meaningful HTTP 404 responses could be returned instead of generic errors.

13.5 Frontend and Backend Integration

Connecting the HTML, CSS and JavaScript frontend with the Spring Boot REST APIs required correct API URLs, request formats, and response handling.

13.6 Testing Different Scenarios

Testing successful CRUD operations as well as invalid inputs and non-existing resources helped identify and resolve implementation issues.

13.7 Learning New Technologies

Understanding Spring Boot, Spring Data JPA, Hibernate, REST APIs, and their integration with MySQL was an important learning experience during the project.

14. FUTURE ENHANCEMENTS

EcoTrack can be further enhanced to provide a more complete campus sustainability management platform.

14.1 User Authentication

User login and role-based access can be added for students, staff, and administrators.

14.2 Admin Dashboard

An administrator dashboard can be introduced to monitor reports, assign issues, and track resolution progress.

14.3 Image Upload

Users can upload images of sustainability issues to provide additional information about reported problems.

14.4 Notifications

Email or application notifications can be implemented to inform users about report status updates.

14.5 Advanced Analytics

Charts and analytics can be added to identify frequently reported categories, locations, and resolution trends.

14.6 Mobile Application

A mobile application can be developed to allow users to report campus sustainability issues directly from their smartphones.

14.7 Deployment

The application can be deployed to a cloud environment so that students and staff can access the system remotely.

15. INSTALLATION AND SETUP

The EcoTrack application can be installed and executed in a local development environment using Java, Spring Boot, MySQL, and Maven.

15.1 Prerequisites

The following software and tools are required:

- ☐ Java Development Kit (JDK)
- ☐ IntelliJ IDEA
- ☐ MySQL Server
- ☐ MySQL Workbench
- ☐ Postman
- ☐ Git and GitHub

15.2 Database Setup

First, MySQL Server is installed and started.

The EcoTrack database is created using MySQL:

```
CREATE DATABASE ecotrack_db;
```

The application is configured to connect to the MySQL database using the Spring Boot application properties.

15.3 Project Setup

The EcoTrack Spring Boot project is opened in IntelliJ IDEA.

The required Maven dependencies are configured in the project, including:

- ☐ Spring Web
- ☐ Spring Data JPA
- ☐ MySQL Driver
- ☐ Spring Boot Validation

Maven automatically downloads the required dependencies when the project is built.

15.4 Running the Application

The main Spring Boot application class is executed from IntelliJ IDEA.

The application starts on the local server at:

<http://localhost:8080>

15.5 API Testing Setup

Postman is used to test the REST API endpoints.

The main API base URL is:

<http://localhost:8080/api/reports>

The API can be tested using HTTP methods such as POST, GET, PUT, and DELETE.

15.6 Database Verification

MySQL Workbench is used to verify the data stored by the application.

The following commands can be used to view the EcoTrack records:

```
USE ecotrack_db;
```

```
SELECT * FROM eco_reports;
```

This confirms that the data submitted through the application is successfully stored in the MySQL database.

16. GIT AND GITHUB / VERSION CONTROL

Git was used as the version control system for maintaining the EcoTrack project source code. GitHub provides a remote repository for storing and managing the project.

16.1 Version Control

Git helps track changes made during development and provides a history of project modifications. It also helps maintain different versions of the source code.

16.2 GitHub Repository

The EcoTrack source code can be maintained in a GitHub repository containing the Spring Boot backend, frontend files, database-related information, configuration files, and project documentation.

16.3 Repository Management

Meaningful commit messages can be used to track major development stages such as:

- Initial project setup
- Backend and database integration
- CRUD REST API implementation
- Frontend development
- Validation and exception handling
- API testing
- Documentation

Sensitive information such as database passwords should not be committed to the public repository.

17. CONCLUSION

EcoTrack successfully demonstrates the development of a complete CRUD-based web application for managing campus sustainability issues.

The application provides a user-friendly frontend for creating and managing reports and a Spring Boot REST API for handling backend operations. Spring Data JPA and Hibernate provide database interaction with MySQL.

The project implements Create, Read, Update, and Delete operations along with search, filtering, server-side validation, and exception handling. API testing was performed using Postman, and database records were verified using MySQL Workbench.

Overall, EcoTrack provides a practical approach to reporting and managing sustainability issues within a campus environment. The project also provided practical experience in frontend development, Java Spring Boot, REST APIs, database management, validation, testing, and version control.

18. REFERENCES

1. Spring Boot Documentation – Official documentation for Spring Boot application development.
2. Spring Data JPA Documentation – Official documentation for database persistence and ORM.
3. MySQL Documentation – Official reference documentation for MySQL database management.
4. Postman Documentation – Official documentation for REST API development and testing.
5. GitHub Documentation – Official documentation for repository and version control management.
6. MDN Web Docs – Reference documentation for HTML, CSS and JavaScript.

